Build and debug, together in space.

A shared engineering workspace where people and AI agents talk, inspect the same live evidence, and shape software while it is being built.

The problem

Developers can delegate more work to agents, but following and steering that work can still require switching between conversations, source files, terminals, traces and dashboards. A finished answer is insufficient when a developer needs to interrupt an investigation, challenge a hypothesis or understand a fix before it runs. Kineframe explores a shared place for that collaboration.

The experience

A developer and an agent share a persistent workspace containing the running application, its relevant source, system relationships, test evidence and proposed changes. Pointing establishes shared focus; conversation communicates intent; direct manipulation helps compare, arrange and inspect artifacts. Desktop and VR views participate in the same session. Observable actions and evidence remain distinct from hypotheses.

One concrete session

1. Observe. Reproduce a checkout failure and examine the actual requests and logs.
2. Investigate. The agent reads relevant files and runs checks; the developer points to a dependency and redirects it.
3. Review. A proposed patch appears alongside the failure it is intended to address.
4. Verify. Apply the reviewed change in an isolated environment, rerun the failing case and compare results.
5. Carry context forward. Keep approved preferences and engineering decisions for later sessions.

Who might use it first

Initial customer hypotheses are developers debugging agent-generated applications, platform engineers investigating multi-service failures, and XR teams building spatial interactions. The platform ambition is broader software engineering. The first validation task is deliberately specific: jointly diagnose and fix one reproducible application bug.

WORKING NAME Kineframe combines movement with a shared frame of reference. Naming is provisional; this brief makes no trademark or domain-availability claim.

An interface for collaboration.

A testable product thesis, with a concrete first implementation.

Technical foundation

A session service maintains versioned source, selected artifacts, conversation, tool activity and test results. Desktop and WebXR clients render that shared state. A tool adapter connects the agent to bounded inspection, execution and patch proposals. Source edits require review; evidence is tied to the revision that produced it. Interruption, stale-change checks and rewind are core behaviors. Existing XR hardware can supply tracking; custom holographic hardware is unnecessary.

Memory that users can inspect

Start with explicit preferences, project decisions and accepted or rejected approaches. Later, repeated corrections could suggest a preference for confirmation. Do not assume every movement reveals intent. Separate interaction calibration from project knowledge and personal preferences; provide scope, provenance, editing and deletion. Passive behavioral learning is a research direction, not an implemented feature.

Prior work and the opportunity

The broad concept has precedents: Cursor supports visual agent instructions [1]; DreamCodeVR changes a live VR application through speech-generated code [2]; Meta provides agent-accessible XR inspection and testing [3]; SPHERE studies persistent spatial preferences [4]. Kineframe's proposed differentiation is the complete shared build-and-debug loop: synchronized attention, inspectable changes, interruption, verification and memory. Its productivity advantage remains to be demonstrated.

What the prototype implements

A browser engineering workspace and WebXR room share a live session. An isolated Node service reproduces a connection leak or timeout mismatch using real HTTP requests. Bounded source patches repair those cases or add a health endpoint; review, apply, verification and rewind are supported. Activity streams as it happens. Hosted AI is optional; a labeled deterministic playbook works without a key. Voice is turn-based, memory is explicit, and headset behavior needs device testing. Arbitrary repositories, production cloud access and autonomous deployments are outside this version.

Validation and business hypothesis

Test the same debugging task against a conventional coding-agent workflow. Measure time to a verified fix, corrective instructions, missed errors, task understanding and repeat use. Compare desktop and VR before claiming an immersion advantage. A possible business model is paid engineering seats plus execution usage, followed by team integrations. Demand, willingness to pay and retention are not yet established.

[1] Cursor Design Mode | [2] DreamCodeVR, UCL
[3] Meta Immersive Web SDK | [4] SPHERE, research paper